Auditing SHAP Explanations Against Latent Failure Mechanisms in Predictive Maintenance

Koshik Debanath
Independent Researcher
koshik.debanath@gmail.com

Abstract

Analysts use SHAP (SHapley Additive exPlanations) to explain predictive maintenance (PdM) classifiers, but rarely test whether an explanation matches the physical cause of failure. Such a test needs ground truth about why each failure happened, which PdM data rarely provides, and we show that an obvious test for it certifies a model fitted to shuffled labels. We use the AI4I 2020 dataset, whose failure label is the logical OR of separately recorded failure mechanisms. We train a classifier on the combined label only, compute out-of-sample SHAP values by cross-fitting, and test two statistics per mechanism against 1,000 models retrained on permuted labels: the share of attribution on the mechanism's trigger features, and how much larger that share is on the mechanism's rows than on other failures. With Holm correction, the tuned XGBoost model (test F1 0.90) passes both tests for heat dissipation, power, and overstrain failure in all five seeds (adjusted $p \leq 0.02$), but not for tool wear failure, which passes only when SHAP is computed on training rows. A graded shortcut experiment shows that the audit flags a synthetic sensor once it carries about a third of the attribution on the affected mechanism's rows, but not at 11%, while test F1 stays between 0.85 and 0.92.

Keywords: explainable AI, SHAP, predictive maintenance, faithfulness evaluation, model randomization, cyber-physical systems

1 Introduction

Predictive maintenance (PdM) systems predict equipment failure from sensor readings of cyber-physical systems, where a wrong reason for a correct alarm can send a technician to the wrong component. Users typically explain these models with post-hoc attributions such as SHAP [15] and read the result as evidence that the model relies on sensible inputs. The explanation itself is rarely tested against the actual cause of failure: a classifier with high accuracy can still rely on a correlate of failure, and a plausible SHAP plot does not reveal this [8].

Testing an explanation against causes requires knowing, for each failure, which mechanism produced it, and industrial datasets seldom record this. A second difficulty is statistical. Rows that belong to a failure mechanism are extreme in that mechanism's features, and we find that a test which ignores this effect passes a model fitted to shuffled labels (Section 6.3).

The AI4I 2020 dataset [16] records the mechanism of each failure. Its target, `Machine failure`, is the logical OR of five flags: tool wear failure (TWF), heat dissipation failure (HDF), power failure (PWF), overstrain failure (OSF), and random failure (RNF). The first four follow published trigger rules on the sensor variables (Table 1); RNF does not. We train on the combined label only and keep the flags for an audit. For each mechanism, we measure how much of the model's out-of-sample SHAP attribution falls on the features its trigger rule uses, and whether that concentration is specific to its own rows, against models retrained on permuted labels [1].

We test three hypotheses:

- **H1:** for each physical mechanism, the model's attribution concentrates on the trigger features (1) more than permuted-label models do and (2) more on that mechanism's rows than on other failures;
- **H2:** the audit flags a model that relies on a shortcut sensor for the mechanism the shortcut imitates, while mechanisms that passed before keep passing;
- **H3:** a naive null, which compares random failure rows under the same model, does not separate a trained model from a permuted-label one.

H1 holds for HDF, PWF, and OSF but not for TWF (Fig. 1); H2 holds once the shortcut carries about a third of the attribution; H3 holds.

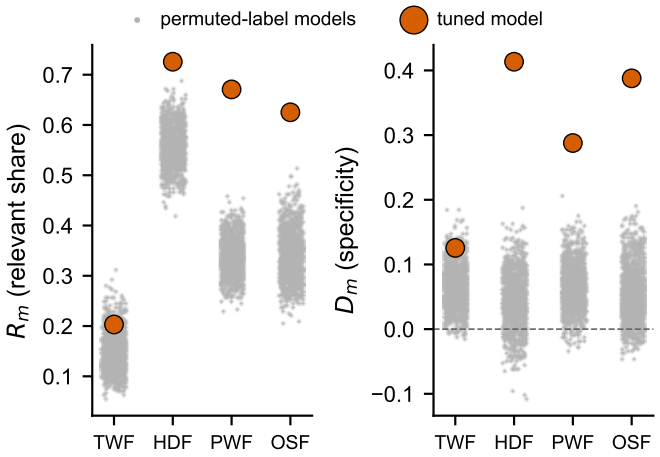


Figure 1: Out-of-sample audit statistics of the tuned model (orange) against 1,000 permuted-label models (gray). R_m (Eq. (2)) is the relevant share and D_m (Eq. (3)) its excess on the mechanism's own rows. HDF, PWF, and OSF lie outside the null on both; TWF does not on R_m.

Our contributions are:

- a per-mechanism SHAP audit with two statistics, out-of-sample attributions, a model-randomization null, and multiple-comparison correction (Section 4);
- evidence that a naive within-model null is confounded (Section 6.3);
- a graded shortcut experiment that measures when the audit detects reliance on a spurious sensor (Section 6.4), plus seed, single-mechanism, in-sample, and raw-feature analyses;
- an imbalance-aware benchmark of seven model families on AI4I 2020 with confidence intervals, and a code base that regenerates every number, table, and figure.

The resampling methods, models, explainer, and randomization idea are existing tools; the audit statistics, the null analysis, and the controls are ours.

2 Related Work

Explanations on AI4I 2020. Matzka [16] introduced the dataset with local explanations of individual predictions, and Zhao et al. [22] report global SHAP importance for tree ensembles on it. Neither tests attributions against the mechanism flags or defines a pass/fail criterion, so there is no audit to compare against. We compare predictive performance instead: Tseng [18] reports random forest positive-class F1 of 0.722, and Khemani and Qureshi [13] report LightGBM macro-F1 of 0.814 ± 0.012 under 5-fold cross-validation with resampling inside the folds (Table 6).

Evaluating attributions against ground truth. A line of work tests explainers on data where the important features are known by construction: XAI-Bench [14] and OpenXAI [2] generate synthetic tabular data, BAM [21] and Zhou et al. [23] modify images or text so that a known feature drives the label, and CLEVR-XAI [3] provides ground-truth masks for visual questions. ROAR [12] avoids ground truth by retraining after removing the features an explainer ranks highest. These methods ask whether an explainer reports what the model uses. We ask the complementary question: given TreeSHAP, whose values are exact for tree ensembles, does the model use what the data-generating mechanism uses? The benchmarks above control the data-generating process, which AI4I 2020 does not expose beyond its trigger rules, so we do not run them as baselines. Chen et al. [6] distinguish attributions that are "true to the model" from ones "true to the data"; our audit targets the second property at the level of mechanism subgroups.

Randomization tests. Adebayo et al. [1] show that some saliency methods give nearly the same explanation for a trained model and a randomized one. We apply the same logic to the model rather than the explainer: permuted-label models define what attribution looks like without any relation to the failure mechanisms.

Causal PdM. Dhande et al. [9] fit causal structural models on a CNC failure dataset of similar size to optimize maintenance cost. They replace the correlational model; we keep it and test its explanation.

3 Background and Problem Setting

SHAP. For a model f and input x with d features, SHAP assigns each feature j an attribution $\phi_j(x)$, its Shapley value with respect to the model output, such that $f(x) = \phi_0 + \sum_{j=1}^{d} \phi_j(x)$, where ϕ_0 is the expected output [15]. TreeSHAP computes these values exactly for tree ensembles. We use $|\phi_j(x)|$ as the magnitude of feature j's contribution to the prediction for x.

Problem setting. We observe rows (x_i, y_i, z_i), where x_i is the sensor vector, $y_i \in \{0, 1\}$ the failure label, and $z_i \in \{0, 1\}^M$ holds M mechanism flags. We train f on (x_i, y_i) only. For each mechanism m, domain rules give a set E_m of features the mechanism's trigger depends on. With $S_m = \{i : z_{i,m} = 1\}$ and $F = \{i : y_i = 1\}$, the audit asks whether attributions on S_m concentrate on E_m, and more so than on $F \setminus S_m$. Two assumptions are unusual: E_m must be known, and the flags must exist for some failures even though the model never sees them.

4 Audit Method

For a set of rows S and mechanism m, let $I_{m,j}(S) = |S|^{-1} \sum_{i \in S} |\phi_j(x_i)|$ and define the share of attribution on the trigger features,

$$r_m(S) = \sum_{j \in E_m} I_{m,j}(S) \Big/ \sum_{j=1}^{d} I_{m,j}(S). \tag{1}$$

The audit uses two statistics, the *relevant share* and the *specificity*,

$$R_m = r_m(S_m), \tag{2}$$
$$D_m = r_m(S_m) - r_m(F \setminus S_m). \tag{3}$$

R_m is large when attribution on the mechanism's rows falls on its trigger features; D_m is large when this concentration is specific to those rows rather than common to all failures.

Out-of-sample attributions. Attributions on training rows reflect what the model fitted, not what it generalizes. We therefore cross-fit: we split the data into five stratified folds, train the full pipeline on four, and compute SHAP values for the held-out fold, so that every audited row is explained by a model that never saw it.

Naive null. An obvious test compares R_m with the same statistic on random, equally sized subsets of F under the same model. It passes a model fitted to shuffled labels (Section 6.3). A likely reason is that rows in S_m are extreme in the features of E_m, because the trigger rule selects them by those features, and $|\phi_j|$ tends to be larger at atypical feature values whatever the model has fitted.

Model-randomization null. We rerun the full procedure – features, resampling, hyperparameters, and cross-fitting – K times on permuted labels and compute both statistics for each run. For $T \in \{R_m, D_m\}$ with null values $T^{(k)}$,

$$p = \frac{1 + \#\{k : T^{(k)} \geq T\}}{K + 1}, \tag{4}$$

and we adjust the four physical mechanisms for multiple comparisons with Holm's method, separately for each statistic. A mechanism passes a test if its adjusted p is below 0.05.

Random failures. RNF has no trigger features. We report the share of its single largest feature and whether it is more concentrated than under the null, as a descriptive check rather than a pass/fail test.

5 Experimental Setup

Dataset. AI4I 2020 has 10,000 rows: a quality variant `Type` $\in \{L, M, H\}$, five process variables (air and process temperature, rotational speed, torque, tool wear), the failure label, and the five mechanism flags. 339 rows (3.39%) are failures. The labels depart from the stated OR rule: 24 rows carry several mechanism flags, 9 failures carry none, and 18 of the 19 RNF rows have `Machine failure` $= 0$, which is why we treat RNF descriptively. We audit all flagged rows and, separately, rows with exactly one flag.

Features. We derive four features from the trigger rules without using the flags: *power* (torque $\times$ speed), *temperature difference* (process $-$ air), *overstrain* (tool wear $\times$ torque), and *overstrain ratio* (overstrain over the type-specific limit). The mechanism flags are components of the label, so we exclude them from the features.

Training. We split the data 70/20/10 into training, validation, and test sets, stratified by label, and fit scaling and one-hot encoding on the training set. We compare three oversamplers – SMOTE [5], Borderline-SMOTE [10], and ADASYN [11] – by the validation F1 of a default XGBoost [7] model and use the best, Borderline-SMOTE (Table 8). We train seven model families (Table 2; the tuned XGBoost is the eighth row, not a separate family).

Selection and tuning. We select on the validation set only: XGBoost has the highest validation F1, so we tune it with a 40-iteration randomized search under 5-fold stratified cross-validation,

Table 1: Failure mechanisms in AI4I 2020 [16] and their trigger features E_m; counts as released. The TWF failure point is random within the window.

Mech.	Trigger	E_m	n
TWF	tool wear at a random point in 200–240 min	tool wear	46
HDF	(process − air temp.) $<$ 8.6 K and speed $<$ 1380 rpm	both temps., their difference, speed	115
PWF	torque × speed (rad/s) outside [3500, 9000] W	torque, speed, power	95
OSF	tool wear × torque above 11,000/12,000/13,000 min·Nm for L/M/H	tool wear, torque, overstrain, its ratio	98
RNF	0.1% chance, independent of the process	–	19

Table 2: Model settings. Unlisted hyperparameters are library defaults (scikit-learn 1.9.1, XGBoost 3.4.1, PyTorch 2.14.0); random seed 42.

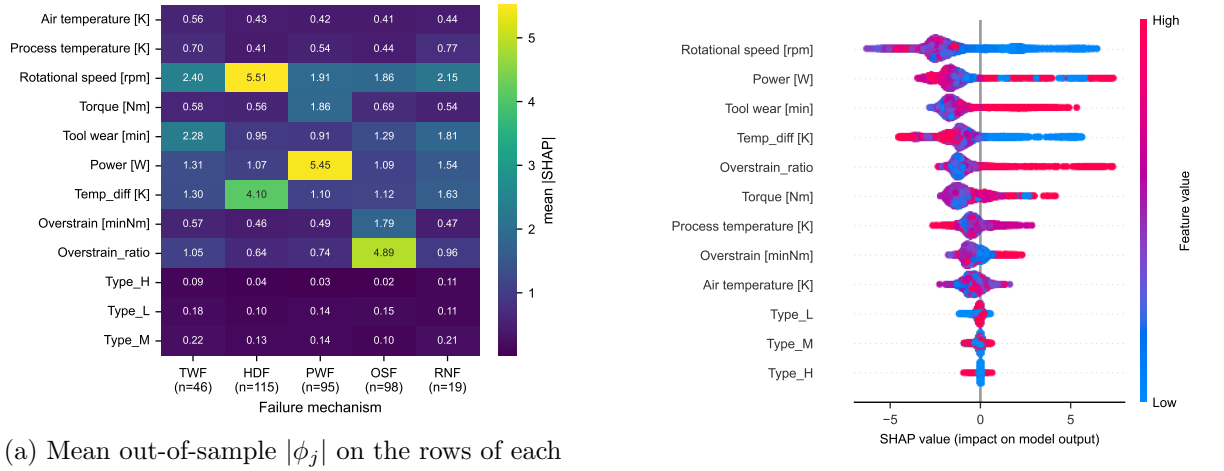
Model	Settings
Logistic regression	max. 1000 iterations
SVM	radial basis function (RBF) kernel, probability outputs
k-NN	$k = 7$
Decision tree	max. depth 10
Random forest	300 trees, max. depth 12
XGBoost	300 trees, max. depth 6, learning rate 0.1
XGBoost (tuned)	200 trees, max. depth 8, learning rate 0.2, subsample 0.8, column sample 1.0, min. child weight 1, $\gamma = 0$
MLP	64 and 32 hidden units, ReLU, batch norm, dropout 0.3/0.2; Adam (learning rate 10^{-3}, weight decay 10^{-5}), binary cross-entropy, batch 64, 40 epochs

scored on F1, over depth 3–8, 200–600 trees, learning rate 0.01–0.2, subsample and column sample 0.6–1.0, minimum child weight 1–5, and γ 0–0.3. We set the decision threshold to maximize F1 on the validation precision-recall (PR) curve. Confidence intervals (CIs) are exact binomial (Clopper–Pearson) for precision and recall and a 5,000-sample stratified bootstrap otherwise. We also run 5-fold stratified cross-validation of the whole pipeline at the default threshold of 0.5.

Audit. The primary audit cross-fits the tuned pipeline with seed 42 and $K = 1{,}000$ permuted-label runs, so the smallest attainable p before correction is 0.001. We repeat it with seeds 1–4 ($K = 200$), with raw features only (without the four derived features, to test whether they make the audit circular), and with SHAP computed on a single model's training rows (*in-sample*). Each run costs $5(K + 1)$ model fits; the runs are independent, and the primary audit takes 11 minutes on eight CPU cores.

Controls. A model trained on permuted labels is exchangeable with the null runs, so its p-values are uniform by construction; we use it only as a sanity check of the code and as the case that the naive null must fail. For H2, we add a synthetic "sensor" $s = z_{\text{HDF}} + \mathcal{N}(0, \sigma^2)$ with no physical role and vary σ from 2 to 0.1. For each σ we report the sensor's share of attribution on HDF rows, test F1, and the audit verdicts ($K = 100$). The sensor is not in E_{HDF}, so a model that relies on it should fail the HDF audit.

6 Results and Discussion



(a) Mean out-of-sample $|\phi_j|$ on the rows of each mechanism

(b) Global SHAP summary on all rows

Figure 2: SHAP attributions of the tuned model. The audit tests the per-mechanism view (a); the global view (b) ranks features overall but does not separate the mechanisms.

6.1 Predictive performance

Table 5 lists test metrics. Tree ensembles outperform the linear and instance-based models. Tuning raises F1 from 0.87 to 0.90, mainly through precision. The test set holds 34 failures, so the uncertainty is large: the tuned model flags 28 rows, all true failures, which gives precision 1.00 (95% CI 0.88–1.00) and recall 0.82 (0.65–0.93). Its scores are calibrated (Brier score 0.0095 against 0.033 for a constant prediction at the base rate; expected calibration error 0.007). Five-fold cross-validation of the full pipeline gives positive-class F1 0.823 ± 0.016 and macro-F1 0.908 ± 0.009,

Table 3: Primary audit (cross-fitted, seed 42, $K = 1{,}000$). Null: mean over permuted-label runs. p: Holm-adjusted over the four physical mechanisms; $^{\times}$ marks $p \geq 0.05$. For RNF the statistic is the largest feature's share and p tests for more concentration than the null.

	n	Top feature	R_m	Null	p	D_m	Null	p
TWF	46	speed	0.203	0.147	$0.096^{\times}$	0.126	0.060	0.039
HDF	115	speed	0.726	0.553	0.004	0.413	0.041	0.004
PWF	95	power	0.671	0.336	0.004	0.288	0.065	0.004
OSF	98	overstrain ratio	0.625	0.339	0.004	0.388	0.054	0.004
RNF	19	speed	0.200	0.221	0.841	–	–	–

Table 4: Holm-adjusted audit p-values for R_m and D_m across analysis variants; $^{\times}$ marks $p \geq 0.05$. The smallest attainable adjusted p is $4/(K+1)$. The permuted-label model is exchangeable with its null and serves only as a sanity check; the last row applies the naive null of Section 4 to that model.

		TWF		HDF		PWF		OSF	
Variant	K	R	D	R	D	R	D	R	D
Primary, seed 42	1000	$0.096^{\times}$	0.039	0.004	0.004	0.004	0.004	0.004	0.004
single-mechanism rows	1000	$0.107^{\times}$	0.044	0.004	0.004	0.004	0.004	0.004	0.004
Seed 1	200	$0.119^{\times}$	$0.080^{\times}$	0.020	0.020	0.020	0.020	0.020	0.020
Seed 2	200	$0.065^{\times}$	0.035	0.020	0.020	0.020	0.020	0.020	0.020
Seed 3	200	$0.154^{\times}$	0.0498	0.020	0.020	0.020	0.020	0.020	0.020
Seed 4	200	$0.199^{\times}$	$0.119^{\times}$	0.020	0.020	0.020	0.020	0.020	0.020
Raw features only	200	$0.289^{\times}$	$0.517^{\times}$	$0.159^{\times}$	0.020	0.020	0.020	0.020	$0.070^{\times}$
In-sample SHAP	1000	0.008	0.004	0.008	0.004	0.004	0.004	0.004	0.004
Permuted labels (sanity)	200	$1.000^{\times}$	$1.000^{\times}$	$0.796^{\times}$	$1.000^{\times}$	$1.000^{\times}$	$0.836^{\times}$	$1.000^{\times}$	$1.000^{\times}$
Permuted labels, naive null	–	0.002	–	0.002	–	0.002	–	0.002	–

against the 0.814 ± 0.012 macro-F1 that Khemani and Qureshi [13] report for LightGBM under a comparable protocol (Table 6). Figs. 3 and 4 show PR curves and the confusion matrix.

Table 5: Positive-class test metrics, sorted by F1; bootstrap 95% CI for F1.

Model	Prec.	Rec.	F1 (95% CI)	ROC-AUC
XGBoost (tuned)	**1.00**	0.82	**0.90** (0.81–0.97)	0.984
XGBoost	0.86	0.88	0.87 (0.78–0.94)	0.988
Random forest	0.79	0.88	0.83 (0.74–0.92)	**0.989**
MLP	0.65	**0.91**	0.76 (0.67–0.84)	0.988
Decision tree	0.68	0.82	0.75 (0.64–0.84)	0.927
SVM	0.42	0.88	0.57 (0.48–0.65)	0.976
k-NN	0.43	0.79	0.56 (0.46–0.65)	0.877
Logistic regression	0.24	0.85	0.37 (0.31–0.43)	0.943

Table 6: Comparison with prior results on AI4I 2020.

Study and protocol	Metric	Value
Random forest, single split [18]	pos.-class F1	0.722
LightGBM, 5-fold CV [13]	macro-F1	0.814 ± 0.012
Ours, single split	pos.-class F1	0.90
Ours, 5-fold CV	pos.-class F1	0.823 ± 0.016
Ours, 5-fold CV	macro-F1	0.908 ± 0.009

6.2 Audit of the tuned model (H1)

For HDF, PWF, and OSF the tuned model passes both tests (Table 3, Fig. 1). On each mechanism's held-out rows, the share of attribution on its trigger features exceeds that of every permuted-label run, and it exceeds the share on other failures by 0.29 to 0.41, against null means below 0.07. The highest-attribution feature is a trigger feature in each case. Fig. 2a shows the attribution matrix.

TWF fails the relevant-share test ($p = 0.096$). Its specificity passes ($p = 0.039$), but its highest-attribution feature out-of-sample is rotational speed, not tool wear. The TWF trigger fires at a random point within a tool-wear window, so tool wear only partly determines it, and the 46 TWF rows give little to generalize from.

6.3 Robustness (H1, H3)

Table 4 repeats the audit. The HDF, PWF, and OSF verdicts hold in all five seeds and on single-mechanism rows. TWF fails the relevant-share test in every seed, and its specificity passes in three of five, so we do not count TWF as supported.

With SHAP computed on training rows, all four mechanisms pass, including TWF ($p = 0.008$). The difference from the cross-fitted result shows that in-sample attributions can overstate how well a model's explanation matches a mechanism.

Without the derived features, PWF passes both tests, HDF passes only on specificity, OSF only on the relevant share, and TWF on neither. The derived features therefore contribute to the audit outcomes, so the circularity concern is only partly resolved; mechanisms defined by a difference or product of inputs are harder to audit when the model must approximate that quantity with splits.

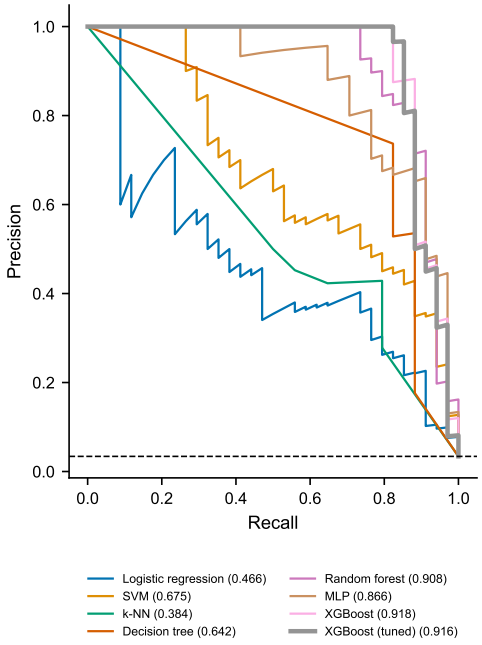


Figure 3: Test PR curves (average precision in parentheses); the dashed line is the 3.4% failure rate.



Figure 4: Confusion matrix of the tuned model on the test set.

The permuted-label model fails every test under the model-randomization null, as it must. The naive null passes the same model on all four mechanisms (Holm-adjusted $p = 0.002$), which supports H3.

6.4 Shortcut sweep (H2)

Table 7: Shortcut sweep ($K = 100$): test F1, the sensor's share of attribution on HDF rows, R_{HDF}, and Holm-adjusted HDF p-values; $^{\times}$ marks $p \geq 0.05$.

σ	F1	Sensor share	R_{HDF}	p_R	p_D	Other mechanisms
2.0	0.89	0.029	0.702	0.040	0.040	all pass
1.0	0.85	0.026	0.699	0.040	0.040	all pass
0.67	0.86	0.032	0.682	0.040	0.040	all pass
0.5	0.90	0.044	0.657	0.040	0.040	all pass
0.35	0.92	0.113	0.572	0.040	0.040	all pass
0.25	0.86	0.360	0.283	$1.000^{\times}$	0.040	fail: TWF
0.15	0.87	0.466	0.158	$1.000^{\times}$	$0.396^{\times}$	fail: TWF
0.1	0.86	0.409	0.206	$1.000^{\times}$	$0.119^{\times}$	fail: TWF

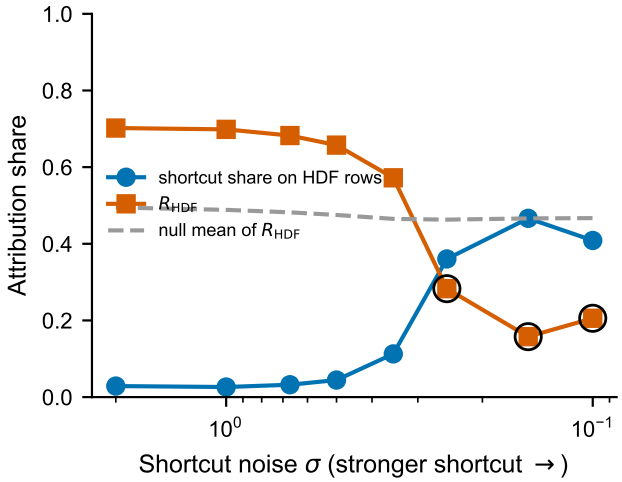


Figure 5: Shortcut sweep. Circles mark noise levels at which the HDF relevant-share test fails.

Table 7 and Fig. 5 show the shortcut sweep. For $\sigma \geq 0.5$ the sensor receives at most 4.4% of the attribution on HDF rows and the audit passes, which is correct because the model barely uses it. At $\sigma = 0.35$ the sensor takes 11% and the audit still passes. From $\sigma = 0.25$, where the sensor takes 36%, the HDF relevant-share test fails, and from $\sigma = 0.15$ the specificity test fails as well. Test F1 stays between 0.85 and 0.92 across the sweep, so accuracy does not reveal the shortcut. H2 therefore holds for substantial reliance on a shortcut but not for light reliance. PWF and OSF pass at every noise level. TWF fails its relevant-share test at the three strongest shortcut levels, consistent with its weak result above.

6.5 Resampling and feature-space structure

Borderline-SMOTE has the highest validation F1 (Table 8). Fig. 6 shows its geometric effect on a logistic regression fitted to a two-dimensional PCA projection: without resampling the decision

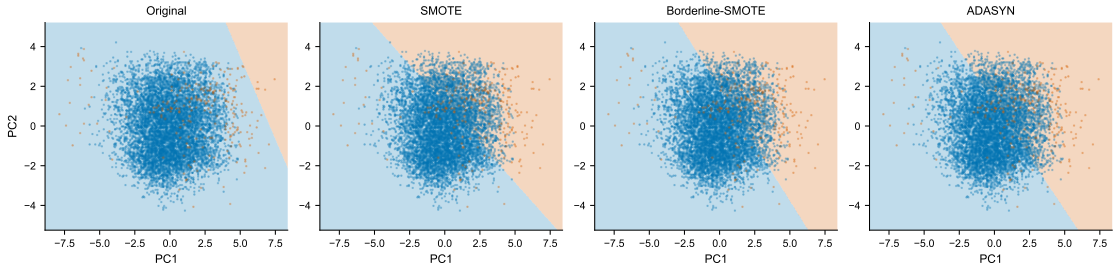


Figure 6: Logistic regression decision boundary on a two-dimensional principal component analysis (PCA) projection of the training data, for each resampler.

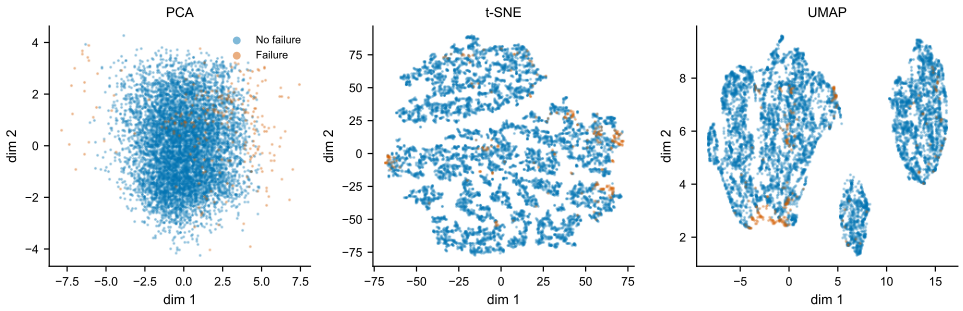


Figure 7: PCA, t-SNE, and UMAP projections of the training data, colored by `Machine failure`.

Table 8: Oversampler comparison on the validation set (default XGBoost).

Sampler	Prec.	Rec.	F1	PR-AUC
Borderline-SMOTE	**0.814**	0.838	**0.826**	0.883
SMOTE	0.699	**0.853**	0.768	**0.892**
ADASYN	0.683	0.824	0.747	0.876

boundary barely reaches the failure class, and each oversampler moves it into the majority class, trading precision for recall.

Fig. 7 projects the training data with PCA, t-distributed stochastic neighbor embedding (t-SNE) [19], and uniform manifold approximation and projection (UMAP) [17]. The classes overlap in the linear projection, consistent with the low F1 of the linear models, while failures form several separate groups in the nonlinear ones, consistent with several distinct mechanisms. The groups persist at t-SNE perplexities from 5 to 100 (repository figure), which matters because t-SNE shapes depend on perplexity [20].

6.6 Limitations

AI4I 2020 is simulated from the same rules we use as ground truth, so the audit checks agreement with the simulator's logic, not with independent measurements of real machines; we contribute a protocol, not a finding about real equipment. Autran et al. [4] show that realistic irregularities added to AI4I 2020 change model performance, so results on the clean data may not transfer.

The audit detects heavy reliance on a shortcut but not light reliance, it needs the trigger features E_m to be known, and the derived features influence its outcomes. We audit one model family with one explainer on one dataset of 10,000 rows; mechanisms with few rows, such as TWF and RNF, give weak evidence either way.

7 Conclusion

We audited SHAP explanations of a PdM classifier against failure mechanisms that the model never saw. With out-of-sample attributions, a model-randomization null, and Holm correction, the tuned model's explanations match the heat dissipation, power, and overstrain mechanisms in all five seeds, but not tool wear failure, which passes only when SHAP is computed on training rows. The audit flags a shortcut sensor once it carries about a third of the attribution, while accuracy stays unchanged. A naive null that compares random failure rows under the same model passes a permuted-label model, so subgroup attribution tests need a null that accounts for how atypical the audited rows are.

The audit applies wherever maintenance records log a root cause for failures that a model predicts as a binary risk; we have not measured how common such records are. Next steps are real maintenance logs with recorded root causes, other explainers and model families, and a formal power analysis of the shortcut threshold.

Acknowledgment

Portions of the manuscript text and the analysis code were drafted with the assistance of an AI system (Claude, Anthropic). The author reviewed and verified all content and takes full responsibility for it. Code, model artifacts, and scripts that regenerate every number, table, and figure are available at `https://github.com/kowshik24/glowing-broccoli`.

References

[1] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. Sanity checks for saliency maps. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[2] Chirag Agarwal, Satyapriya Krishna, Eshika Saxena, Martin Pawelczyk, Nari Johnson, Isha Puri, Marinka Zitnik, and Himabindu Lakkaraju. OpenXAI: Towards a transparent evaluation of model explanations. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 35, 2022.

[3] Leila Arras, Ahmed Osman, and Wojciech Samek. CLEVR-XAI: A benchmark dataset for the ground truth evaluation of neural network explanations. *Information Fusion*, 81:14–40, 2022. doi: 10.1016/j.inffus.2021.11.008.

[4] J.-V. Autran, V. Kuhn, J.-P. Diguet, M. Dubois, and C. Buche. AI4I-PMDI: Predictive maintenance datasets with complex industrial settings' irregularities. *Procedia Computer Science*, 246:1201–1209, 2024. doi: 10.1016/j.procs.2024.09.546.

[5] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16: 321–357, 2002.

[6] Hugh Chen, Joseph D. Janizek, Scott Lundberg, and Su-In Lee. True to the model or true to the data? *arXiv preprint arXiv:2006.16234*, 2020.

[7] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[8] Logan Cummins, Alexander Sommers, Somayeh Bakhtiari Ramezani, Sudip Mittal, Joseph Jabour, Maria Seale, and Shahram Rahimi. Explainable predictive maintenance: A survey of current methods, challenges and opportunities. *IEEE Access*, 12:57574–57602, 2024. doi: 10.1109/ACCESS.2024.3391130.

[9] Shaunak Dhande, Chutian Ma, Giacinto Paolo Saggese, Paul Smith, and Krishna Taduri. A benchmark of causal vs. correlation AI for predictive maintenance. *arXiv preprint arXiv:2512.01149*, 2026.

[10] Hui Han, Wen-Yuan Wang, and Bing-Huan Mao. Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning. In *International Conference on Intelligent Computing*, pages 878–887. Springer, 2005.

[11] Haibo He, Yang Bai, Edwardo A. Garcia, and Shutao Li. ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE International Joint Conference on Neural Networks*, pages 1322–1328. IEEE, 2008.

[12] Sara Hooker, Dumitru Erhan, Pieter-Jan Kindermans, and Been Kim. A benchmark for interpretability methods in deep neural networks. In *Advances in Neural Information Processing Systems*, volume 32, pages 9737–9748, 2019.

[13] Kushal Khemani and Anjum Nazir Qureshi. AI-driven predictive maintenance for connected vehicles using environmental context integration evaluated through simulation benchmarking and field validation. *Discover Vehicles*, 2:19, 2026. doi: 10.1007/s44465-026-00023-2.

[14] Yang Liu, Sujay Khandagale, Colin White, and Willie Neiswanger. Synthetic benchmarks for scientific research in explainable machine learning. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[15] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 2017.

[16] Stephan Matzka. Explainable artificial intelligence for predictive maintenance applications. In *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)*, pages 69–74. IEEE, 2020. doi: 10.1109/AI4I49448.2020.00023.

[17] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[18] Chun-Jan Tseng. A physics-informed benchmarking framework for machine learning and tree-based ensembles in IIoT-enabled predictive maintenance. *Sensors*, 26(16):5026, 2026. doi: 10.3390/s26165026.

[19] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.

[20] Martin Wattenberg, Fernanda Viégas, and Ian Johnson. How to use t-SNE effectively, 2016. URL `https://distill.pub/2016/misread-tsne/`. Distill.

[21] Mengjiao Yang and Been Kim. Benchmarking attribution methods with relative feature importance. *arXiv preprint arXiv:1907.09701*, 2019.

[22] Wen Zhao, Jiawen Ding, Xueting Huang, and Yibo Zhang. Research on milling machine predictive maintenance based on machine learning and SHAP analysis in intelligent manufacturing environment. *arXiv preprint arXiv:2512.01205*, 2025.

[23] Yilun Zhou, Serena Booth, Marco Tulio Ribeiro, and Julie Shah. Do feature attribution methods correctly attribute features? In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9623–9633, 2022.